Solar Chromosphere Velocity (v2)

Method and Implementation Manual

Program flow, streaming framework, spectral fitting and the velocity model

`solar-chromosphere-velocity` v2.2

November 6, 2026

Abstract

This manual documents the method and implementation of `solar-chromosphere-velocity` v2. The program takes CHASE/HIS level-1 spectral cubes, inverts the line-of-sight (LOS) velocity field, and decomposes it into components with distinct physical meaning: differential rotation, a limb shift, meridional circulation and a systemic translational velocity. The document covers the program flow and its timed stages, the streaming read plus batched-FFT framework, the line-centre fit and the cross-correlation Doppler inversion, the complete velocity model together with the physical meaning and explicit expression of every term, parameter estimation and identifiability, the output contract, and the measured speed-up over v1.0.1. Every equation is tied to the code that implements it, and example products for both Hα and Fe I are included in Section 12.

Contents

1 Introduction and scope **2**
- 1.1 Input . . . 2
- 1.2 Output . . . 2
- 1.3 Scope and limitations . . . 2

2 Program flow **3**
- 2.1 Stages . . . 3
- 2.2 Batch mode and naming . . . 4

3 Reading and the streaming strategy **4**

4 Solar geometry **4**

5 Spectral line-centre fitting **5**
- 5.1 The wavelength solution as a hint . . . 5
- 5.2 Gaussian refinement . . . 5

6 Doppler velocity inversion **5**
- 6.1 Correlation window . . . 5
- 6.2 Cross-correlation and sub-pixel refinement . . . 6
- 6.3 Velocity calibration . . . 6

7 The velocity model **6**
7.1 The model equation . . . 6
7.2 Physical meaning of each term . . . 7
7.3 Degenerate cases . . . 7
7.4 Layers derived from the model . . . 8

8 Parameter estimation, uncertainty and identifiability **8**
8.1 Linear least squares and streaming normal equations . . . 8
8.2 Scanning N . . . 8
8.3 The $e \geq 0$ active-set step . . . 8
8.4 Uncertainty . . . 9
8.5 Identifiability and flags . . . 9

9 Streaming and parallel framework **9**

10 Output structure **9**

11 Differences from v1.0.1 and the measured speed-up **10**

12 Example products **10**

13 The joint Hα + Fe I product **11**

A Symbols **14**

B Key configuration options (v2.1 names) **14**

1 Introduction and scope

1.1 Input

CHASE (Xihe) HIS level-1 spectral cubes, typically of shape (wavelength, y, x) = (118, 2313, 2304) for Hα (6562.82 Å, `SPECLINE='HA'`) or (46, 2313, 2304) for Fe I (6569.22 Å, `SPECLINE='FEI'`). The cube is normally stored RICE_1 tiled-compressed in `hdu[1]`, with the simultaneous 2-D line-core intensity image in HDU 0. Inputs that are already 2-D LOS velocity maps (for example the `*_Vdop.fits` products written by v1.0.1) are also accepted.

1.2 Output

One multi-extension FITS file per observation time, containing ten 2-D layer extensions plus a parameter table, together with an `index.csv` observation log and optionally multi-panel PNG figures. Outputs are **never** written next to the inputs.

1.3 Scope and limitations

- Physical assumptions: a single absorption line, an LOS velocity that is linear in the line-centre displacement, and a large-scale flow field described over a full-disk image by the nine-parameter analytic model of Section 7.

- The method returns a *relative* LOS velocity. The inversion template is the average profile of the disk-centre region, so any absolute (gravitational plus convective) line shift is already contained in the template. The parameter G is a systemic shift relative to that template and **must not be read as a gravitational redshift**.
- The method is not intended for local flows in strong active regions, nor for multi-component profiles (for example Hα with pronounced wing emission).

2 Program flow

2.1 Stages

`process_file()` in `processor.py` splits one file into the timed stages listed in Table 1; each stage records its wall time in `ProcessResult.timings`.

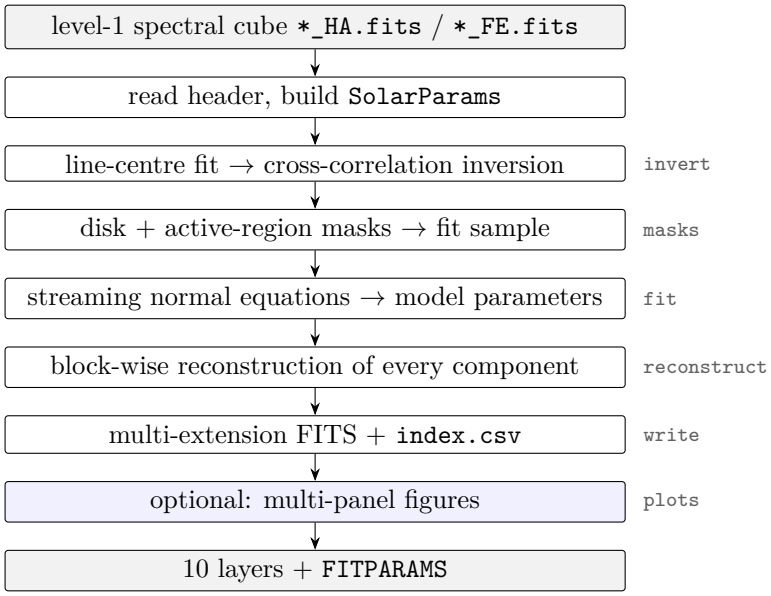


Figure 1: Per-file processing flow. The small labels on the right are the keys written into `timings`.

The input type is decided before the flow starts, by `data_loader.detect_data_type()`: a 3-D cube goes through the spectral inversion, a 2-D velocity map goes straight to the model fit. For 2-D input the `BUNIT` is honoured (`m/s` is converted to `km/s`); a map whose unit is neither a velocity nor recognisable is refused by default, so that an intensity image cannot silently be fitted as if it were a velocity.

Table 1: Timed stages (`ProcessResult.timings`).

key	content	main code
`read+invert`	read plus inversion	`processor._read_input`
`invert`	inversion only	`inversion.DopplerInverter.run`
`masks`	disk and active-region masks	`masks.active_region_mask`
`fit`	least-squares model fit	`models.fit_model_streaming`
`reconstruct`	component and error surfaces	`processor._reconstruct_blocks`
`write`	FITS and log	`output.write_multihdu`
`plots`	multi-panel figures (optional)	`plotting`
`total`	single file, total	—

2.2 Batch mode and naming

`process_directory()` calls `process_file()` for each input and writes one row per input into `index.csv`, including failed inputs. The output path is determined by the **observation time in the header**:

```
<output_dir>/<YYYYMMDD>/<YYYYMMDD>T<HHMMSS>_<LINE>_scv.fits
```

The time is taken from `STR_TIME`, falling back to `DATE-OBS`, then to a time parsed from the file name or its modification time. Which one was used is recorded in the `source` column of `index.csv` and in the `TIMEKEY` header card. Different observation times therefore **cannot overwrite each other**, independently of the file names. If the target name already exists and `overwrite_existing` is off, the program adds a suffix rather than overwriting silently.

3 Reading and the streaming strategy

A RICE_1 tiled-compressed FITS cube is a `CompImageHDU` in astropy. Touching `hdu.data` decompresses the entire cube into memory, whereas `hdu.section[...]` decodes only the requested tiles. v2 uses the latter throughout and advances in strips of `spatial_strip_rows` rows (default 32):

$$\text{cube } (n_\lambda, n_y, n_x) \;\longrightarrow\; \underbrace{(n_\lambda^{\text{win}},\, s,\, n_x)}_{\text{one strip}}, \qquad s = \texttt{spatial_strip_rows}. \tag{1}$$

The strip height is a measured compromise: on a 2313×2304 RICE cube, 32 rows gave both the smallest peak memory and the shortest inversion time against 64, 128, 256 and 512 rows. The strategy makes memory essentially independent of the image area and proportional only to the strip area.

4 Solar geometry

For every pixel, the offset from disk centre is first rotated by the instrument roll angle P_0 (header `INST_ROT`):

$$\begin{pmatrix} x_r \\ y_r \end{pmatrix} = \begin{pmatrix} \cos P_0 & -\sin P_0 \\ \sin P_0 & \cos P_0 \end{pmatrix} \begin{pmatrix} x - x_c \\ y - y_c \end{pmatrix}, \qquad r = \sqrt{x_r^2 + y_r^2}, \tag{2}$$

where (x_c, y_c) is the disk-centre pixel and $R_\odot$ is the solar radius in pixels (header `R_SUN`). Assuming an orthographic projection, set $\sin\rho = r/R_\odot$ and $\rho = \arcsin(r/R_\odot)$; with the position angle θ defined by $\cos\theta = y_r/r$ and $\sin\theta = -x_r/r$, the heliographic coordinates follow from

$$\sin B = \sin B_0 \cos\rho + \cos B_0 \sin\rho \cos\theta, \tag{3}$$

$$\sin L = -\,\frac{\sin\rho\,\sin\theta}{\cos B}, \tag{4}$$

where B is the heliographic latitude, L the longitude, and B_0 the disk-centre latitude (header `B0`). At the disk centre $r = 0$ the position angle is undefined; the implementation uses the $r \to 0$ limit ($\cos\theta = 1$, $\sin\theta = 0$). The routine also returns $\mu = \cos\rho$, the limb-darkening geometrical factor.

Code: `geometry.solar_coords()`, with the batch version `geometry.coord_arrays()`. The whole pipeline uses the vectorised form; it measured about 59× faster than v1's per-pixel scalar loop.

5 Spectral line-centre fitting

The first inversion step is to establish which wavelength sample holds the line centre. The program takes the mean profile inside `reference_profile_radius` pixels of disk centre (default 100) as the reference profile $\bar{I}(j)$ and locates the centre on it.

5.1 The wavelength solution as a hint

If the header contains `CDELT3`, `CRVAL3` and `WAVE_LEN` (or a recognisable `SPECLINE`), the linear wavelength solution

$$\lambda(i) = \mathrm{CRVAL3} + \big(i - (\mathrm{CRPIX3} - 1)\big)\,\mathrm{CDELT3} \tag{5}$$

can be inverted for the line-centre sample,

$$i_\star = \frac{\lambda_0 - \mathrm{CRVAL3}}{\mathrm{CDELT3}} + (\mathrm{CRPIX3} - 1), \tag{6}$$

with λ_0 the rest wavelength from `WAVE_LEN` or looked up from `SPECLINE`. **This is only a hint**: in the level-1 data examined here the wavelength solution and the data themselves disagree by about 1.4 pixels ($\sim 3\,\mathrm{km\,s^{-1}}$), so the centre must be measured from the data.

5.2 Gaussian refinement

Around the local minimum of $\bar{I}$ the program fits

$$I(i) = I_c + A \exp\left[-\frac{(i - i_0)^2}{2\sigma^2}\right] \tag{7}$$

by least squares, giving the continuous line centre i_0 (`LineCenter.index` is the rounded integer, `continuous` is i_0). With `-legacy` the v1 threshold method (`linecenter._legacy_center`) is used instead, to reproduce v1.

6 Doppler velocity inversion

6.1 Correlation window

With line centre i_0 and half-width $w =$ `correlation_half_width` (default 10), the window is $[i_0 - w,\ i_0 + w]$ of length $L = 2w + 1$. The reference signal is the average profile inside that window,

$$T(j) = \bar{I}(i_0 - w + j), \qquad j = 0, \ldots, L - 1. \tag{8}$$

When continuum subtraction is enabled (`continuum_subtraction`, on by default outside legacy mode) the mean of each window is removed from both the reference and the measured signal:

$$\tilde{T}(j) = T(j) - \bar{T}, \qquad \tilde{S}_p(j) = S_p(j) - \bar{S}_p. \tag{9}$$

Removing the window mean suppresses the bias that a continuum slope would otherwise introduce into the peak position.

6.2 Cross-correlation and sub-pixel refinement

For each spatial pixel p with observed window $S_p(j)$, the circular cross-correlation is

$$C_p(k) = \sum_{j=0}^{L-1} \tilde{S}_p(j)\, \tilde{T}(j-k), \tag{10}$$

evaluated for an entire strip at once through the FFT,

$$C_p = \mathrm{IFFT}\Big[\mathrm{FFT}(\tilde{S}_p) \cdot \overline{\mathrm{FFT}(\tilde{T})}\Big], \tag{11}$$

followed by `fftshift` to place zero lag at $k = \lfloor L/2 \rfloor$.

With the peak at $k^\star = \arg\max_k C_p(k)$, a three-point parabola gives the sub-pixel refinement

$$x = \frac{C_p(k^\star - 1) - C_p(k^\star + 1)}{2\big[C_p(k^\star - 1) - 2C_p(k^\star) + C_p(k^\star + 1)\big]}, \tag{12}$$

so the displacement relative to the window centre, in wavelength samples, is

$$\delta_p = (k^\star + x) - \Big\lfloor \frac{L}{2} \Big\rfloor. \tag{13}$$

If $k^\star$ sits on a window edge the parabolic term is dropped and the integer peak is used, which avoids extrapolating beyond the window. Outside legacy mode x is clipped to $[-0.5,\, 0.5]$ (`limit_subpixel_shift`).

6.3 Velocity calibration

From the dispersion CDELT3 (Å/pixel) and the rest wavelength λ_0, one sample corresponds to

$$\kappa \;=\; \frac{c\,\mathrm{CDELT3}}{\lambda_0}, \qquad c = 299792.458 \ \mathrm{km\,s^{-1}}, \tag{14}$$

hence the LOS velocity

$$\boxed{V_{\mathrm{LOS}}(p) \;=\; \delta_p \cdot \kappa} \tag{15}$$

For Hα (CDELT3 $= 0.04844$ Å, $\lambda_0 = 6562.82$ Å) this gives $\kappa \approx 2.2139\,\mathrm{km\,s^{-1}}$/pixel. With `-legacy` the v1 hard-coded constant $\kappa_{\mathrm{v1}} = 1.1068 \times 2 = 2.2136\ \mathrm{km\,s^{-1}}$/pixel is used instead. Off-limb pixels ($r > R_\odot$) are set to 0 following v1 semantics, and pixels beyond `disk_mask_radius` $\cdot\, R_\odot$ are set to `NaN` so that they take no part in the fit. The sign convention is **redshift (receding) positive**, recorded in the `UNITSYS` card.

7 The velocity model

7.1 The model equation

Write B for heliographic latitude (with $s_B \equiv \sin B$), L for longitude and ρ for the heliocentric angle, and define the two LOS projection factors

$$\mathcal{P} = \cos B\, \sin L\, \cos B_0, \tag{16}$$

$$\mathcal{M} = \cos L\, \sin B\, \cos B_0 \;-\; \cos B\, \sin B_0. \tag{17}$$

$\mathcal{P}$ is the LOS projection of the rotation velocity (directed along longitude) and $\mathcal{M}$ is the LOS projection of the meridional circulation after expanding it on the two unit vectors $(\hat{B}, \hat{L})$. The model LOS velocity is then

$$\begin{aligned} V_{\rm model}(B, L, \rho) = s\Big(A + Bs_B^2 + Cs_B^4 + D\sin^{10}\rho\Big)\mathcal{P} \quad & \text{(differential rotation)} \\ + \; e\,(1-\cos\rho)^N \quad & \text{(limb shift)} \\ + \; (\mathrm{MC1}\, s_B + \mathrm{MC2}\, \sin 2B)\mathcal{M} \quad & \text{(meridional circulation)} \\ + \; G \quad & \text{(systemic translation)} \end{aligned} \tag{18}$$

where $s = R_\odot = 6.957 \times 10^5\,\mathrm{km}$ when the rotation parameters are given as angular rates (`rotation_amplitude='rad_per_s'`), and $s = 1$ otherwise (velocity amplitudes in km/s).

7.2 Physical meaning of each term

Table 2 lists the nine parameters, what each denotes and its complete expression. The parameter order `TERM_ORDER` $= (A, B, C, D, G, e, N, \mathrm{MC1}, \mathrm{MC2})$ is the same as in v1, so historical parameter files can be compared directly.

Table 2: Model parameters, physical meaning and expression. All amplitudes are in km/s; N is dimensionless.

parameter	physical meaning	expression
A	equatorial rotation amplitude (constant term of the Ω expansion)	$s\,A\,\mathcal{P}$
B	rotation, $\sin^2 B$ term (differential rotation)	$s\,B\,s_B^2\,\mathcal{P}$
C	rotation, $\sin^4 B$ term	$s\,C\,s_B^4\,\mathcal{P}$
D	rotation, $\sin^{10}\rho$ limb term	$s\,D\,\sin^{10}\rho\,\mathcal{P}$
e	limb-shift amplitude (`limb_amplitude_positive` constrains $e \geq 0$)	$e\,(1-\cos\rho)^N$
N	power-law exponent of the limb shift (chosen on a grid)	$(1-\cos\rho)^N$
MC1	meridional circulation, $\sin B$ component	$\mathrm{MC1}\,s_B\,\mathcal{M}$
MC2	meridional circulation, $\sin 2B$ component	$\mathrm{MC2}\,\sin 2B\,\mathcal{M}$
G	systemic translational velocity (global zero point of the model)	G

On G. The inversion template is the disk-centre average profile, so G absorbs three things at once: a genuine systemic velocity, the offset between that template and the rest wavelength, and the absolute wavelength zero point of the instrument. It is mathematically degenerate with the choice of template, with the zero-point correction `zero_point_km_s`, and with the mean removed by continuum subtraction. It must therefore be read as a *systemic zero-point reading only, never as a gravitational redshift.* For reference the nominal gravitational redshift is $GM/(R_\odot c) \approx 0.636\,\mathrm{km\,s^{-1}}$, whereas measured values of $|G|$ are around 0.014–0.20 $\mathrm{km\,s^{-1}}$ depending on the line and the event; the two are not of the same order.

7.3 Degenerate cases

When a line carries no measurable limb shift over the full disk (as is the case for disk-averaged Hα), the linear least-squares solution may return $e < 0$. If `limb_amplitude_positive` is set, the program then pins e to the boundary $e = 0$ (an active-set step), so $V_{\rm LIMB} \equiv 0$ and N has no influence on the model whatsoever: neither parameter is identifiable. Both are flagged and given `NaN` errors (Section 8.5). For the Fe I data examined here, $e \approx 0.19\,\mathrm{km\,s^{-1}}$ and is significantly positive, so $V_{\rm LIMB}$ shows a genuine limb-brightening ring (Figure 3).

7.4 Layers derived from the model

`models.VelocityModel.components()` produces the layers in Table 3. `V_LARGE` is the sum of the four components (the model surface) and `V_RESID` is the observation minus that surface, with the quiet-Sun mean (header `QUIETMN`) removed as well.

Table 3: Definition of the output layers (`output.LAYERS`).

EXTNAME	physical meaning	expression
VLOS_OBS	observed LOS velocity	$\delta_p\kappa$, Eq. (15)
VLOS_CORR	LOS velocity after the zero-point correction	$V_{\rm LOS}$ − ZEROPT
V_DIFFROT	differential-rotation component	$s(A + Bs_B^2 + Cs_B^4 + D\sin^{10}\rho)\mathcal{P}$
V_LIMB	limb-shift component	$e(1-\cos\rho)^N$
V_MERID	meridional circulation, LOS projected	$(\mathrm{MC1}\,s_B + \mathrm{MC2}\sin 2B)\mathcal{M}$
V_OFFSET	systemic translation	G
V_LARGE	total large-scale flow surface	sum of the four above
V_RESID	residual LOS velocity	VLOS_CORR − V_LARGE − QUIETMN
V_MODEL_ERR	1σ of the model surface	$\sqrt{\boldsymbol{x}^\top\Sigma\boldsymbol{x}}$
MASK_ACTIVE	active-region mask	0 quiet, 1 active

8 Parameter estimation, uncertainty and identifiability

8.1 Linear least squares and streaming normal equations

Except for N, the model is linear in every parameter: for a given N the eight columns of the design matrix $\boldsymbol{X}$ are $[\mathcal{P},\ s_B^2\mathcal{P},\ s_B^4\mathcal{P},\ \sin^{10}\rho\,\mathcal{P},\ (1-\cos\rho)^N,\ s_B\mathcal{M},\ \sin 2B\,\mathcal{M},\ 1]$, with parameter vector $\boldsymbol{\beta} = (A, B, C, D, e, \mathrm{MC1}, \mathrm{MC2}, G)^\top$. Minimising

$$\chi^2(\boldsymbol{\beta}) = \sum_p \left(V_{\rm LOS}^{(p)} - \boldsymbol{x}_p^\top\boldsymbol{\beta}\right)^2 \tag{19}$$

gives the normal equations $\boldsymbol{X}^\top\boldsymbol{X}\,\boldsymbol{\beta} = \boldsymbol{X}^\top\boldsymbol{y}$. The program never forms the full design matrix; it accumulates strip by strip,

$$\boldsymbol{X}^\top\boldsymbol{X} \mathrel{+}= \boldsymbol{X}_b^\top\boldsymbol{X}_b, \qquad \boldsymbol{X}^\top\boldsymbol{y} \mathrel{+}= \boldsymbol{X}_b^\top\boldsymbol{y}_b, \qquad y^\top y \mathrel{+}= \boldsymbol{y}_b^\top\boldsymbol{y}_b, \tag{20}$$

so memory is independent of the pixel count (a 9×9 matrix), and solves the small system by Cholesky factorisation.

8.2 Scanning N

N enters as a power-law exponent and cannot be solved linearly, so the program solves the same normal equations for each value in `limb_exponent_scan` (default $\{1, 1.25, 1.5, 2, 2.5, 3, 4, 5\}$) and takes the value with the smallest residual sum of squares. All grid points share the accumulators, so the extra cost is essentially zero. The resulting N-RSS curve is written into the parameter-table header (`NSCAN`, `NSCAN02`, ...).

8.3 The $e \geq 0$ active-set step

If the unconstrained solution gives $e < 0$, the constrained optimum lies on the boundary $e = 0$: the program **drops the column for** e and re-solves. This necessarily raises the residual sum of squares — that is the price of the constraint, not a reason to reject it.

8.4 Uncertainty

The covariance is $\Sigma = \hat{\sigma}^2(\boldsymbol{X}^\top \boldsymbol{X})^{-1}$ with $\hat{\sigma}^2 = \mathrm{RSS}/(n-p)$. For a dropped column the corresponding row and column of the covariance are zero. The per-pixel model uncertainty written to `V_MODEL_ERR` is

$$\sigma_{\mathrm{model}}(\boldsymbol{x}) = \sqrt{\boldsymbol{x}^\top \Sigma\, \boldsymbol{x}}. \tag{21}$$

8.5 Identifiability and flags

Reporting a numerical error for a parameter that is not identifiable would read as a very precise measurement. For the following cases the program therefore sets the error to `NaN`, writes a `FLAG` in `FITPARAMS`, writes **no** `ERR_*` card for that parameter, and summarises them in the `FITFLAG` card of `PRIMARY`:

- `PINNED_AT_ZERO`: e is constrained to $e = 0$ (active-set step);
- `DEGENERATE_WITH_E`: with e pinned, N has no effect on the model;
- `FLAT_SCAN`: the N scan changes the RSS by a negligible amount;
- `SCAN_GRID`: N is chosen from a discrete grid and never has a covariance-based uncertainty.

Note that the covariance matrix itself is **not** set to `NaN`: only the reported 1σ is. Blanking the covariance would turn the entire `V_MODEL_ERR` layer into `NaN`, because it is computed from $\sqrt{\boldsymbol{x}^\top \Sigma \boldsymbol{x}}$.

9 Streaming and parallel framework

The performance of v2 comes from three layers, in decreasing order of contribution:

1. **Vectorised solar geometry.** (B, L, ρ) is computed for every pixel of a strip in one operation, replacing v1's per-pixel Python loop ($\sim 59\times$).
2. **Batched-FFT cross-correlation.** Eq. (10) is evaluated for all pixels of a strip simultaneously: a single FFT along the wavelength axis of an array of shape $(L,\ s \cdot n_x)$ ($\sim 38\times$; 24.4 µs $\rightarrow$ 0.65 µs per pixel on synthetic data).
3. **Streaming normal equations.** The fit accumulates an 8×8 matrix strip by strip, so its memory does not grow with the number of pixels; the reconstruction is block-wise too, and only the ten output layers remain resident.

The configuration keys `parallel_processing` and `parallel_processes` are **reserved**: single-file processing is already fast (Section 11), and parallelising across *files* with the scheduler is more efficient. The in-process path is deliberately single-threaded so that results are reproducible.

Complexity. With P pixels, window length $L = 2w + 1$ and strip height s: the inversion is $\mathcal{O}(PL \log L)$, the fit is $\mathcal{O}(P \cdot 8^2)$, and the peak memory is $\mathcal{O}(s\, n_x\, L + 10\, n_y n_x)$ — the first term from the strip, the second from the ten float32 output layers.

10 Output structure

One observation time produces one FITS file with a fixed extension order:

`PRIMARY`, `VLOS_OBS`, `VLOS_CORR`, `V_DIFFROT`, `V_LIMB`, `V_MERID`, `V_OFFSET`, `V_LARGE`, `V_RESID`, `V_MODEL_ERR`, `MASK_ACTIVE`, `FITPARAMS`

`PRIMARY` carries the observation metadata (time, source file, disk geometry, `CALSPEC`, `INVMETH`, `UNITSYS`, goodness of fit and `FITFLAG`) and is guaranteed to contain **no duplicate keywords**. Each

velocity layer carries `BUNIT='km/s'`, `QUANTITY`, `FORMULA`, `PARAMS` and `MODELVER`. `FITPARAMS` is a nine-row binary table with columns `NAME, VALUE, ERROR, UNIT, COMPONENT, FORMULA, DESCRIPTION, FLAG`. `index.csv` has one row per input per run, with the observation time, the source of that time, the output file name, the model version, the number of fit points, the RMS, and the nine parameter values.

11 Differences from v1.0.1 and the measured speed-up

Table 4: Main physical and numerical differences.

aspect	v2 relative to v1.0.1
velocity calibration	computed from the header `CDELT3/WAVE_LEN` (Eq. (14)) instead of hard-coded 1.1068×2
line centre	Gaussian fit with the wavelength solution as a hint; v1 used a threshold method
rotation projection	includes the $\cos B_0$ factor; v1 omitted it
meridional circulation	LOS projected via $\mathcal{M}$ (Eq. (17)); v1 did not project
continuum	correlation windows are mean-subtracted; v1 did not subtract
sub-pixel	the parabolic term is clipped to ± 0.5 pixel; v1 did not clip
reading	strip-wise `hdu.section` only; v1 decompressed the whole cube
fitting	streaming normal equations; v1 used per-pixel `curve_fit`
output	one multi-extension FITS per observation time; v1 wrote single-layer files next to the inputs

Table 5: Measured on BlueWhale (2026-04-24 Hα, $2313 \times 2304 \times 118$, 64-core `mpi` queue, conda environment with python 3.11, astropy 8.0.1, numpy 2.4.6), single process.

implementation	wall time (s)	peak memory (GB)
v1.0.1 (original)	169.17	4.266
v2 `-legacy`	8.90	0.575
v2 complete physics	9.59	0.583
speed-up (complete physics)		17.6×
speed-up (`-legacy`)		19.0×
memory ratio		7.3×

In the same run the v2 `VLOS_OBS` differs from the v1 product by $\max|\Delta| = 9.5 \times 10^{-7}\,\mathrm{km\,s^{-1}}$ (rms 7.5×10^{-8}) and the identifiable parameters by $\max|\Delta| = 4.4 \times 10^{-7}$: the two versions describe the same curve, and the differences come from the physical corrections listed above.

Note that `-visualize` is not free: the nine figures roughly double the wall time and raise the peak from 0.58 GB to about 1.5 GB (three figures) or 3.0 GB (nine figures). A plotting failure never affects the FITS product.

12 Example products

The two figures below are real products of this version, drawn by the `-visualize -plot-per-component` option, and are shown here as a reference for what the layer set looks like in practice. In both cases every panel has its own symmetric colour scale, because the components differ by orders

of magnitude, and the axes are in arcseconds relative to disk centre (derived from `CDELT1`/`CDELT2` and the disk centre).

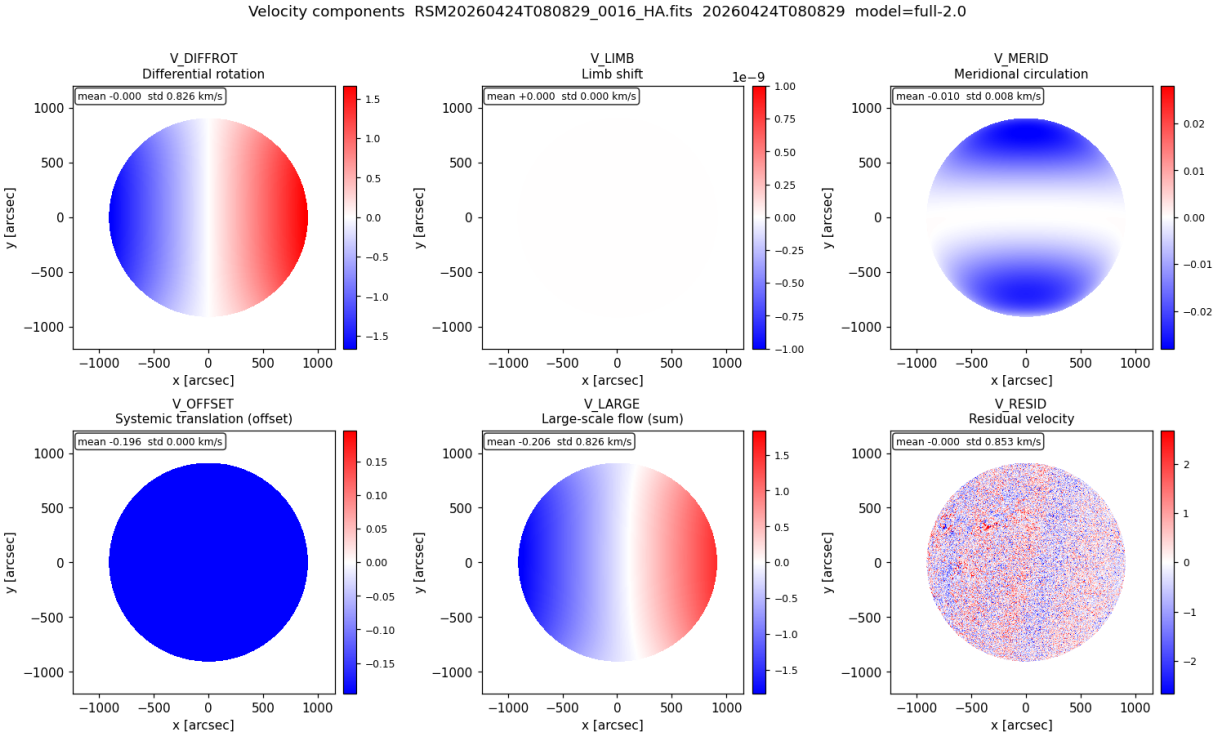


Figure 2: Hα, 2026-04-24 08:08:29 UT (`RSM20260424T080829_0016_HA.fits`). The four model terms, their sum and the residual. `V_DIFFROT` is the expected east–west antisymmetric differential-rotation pattern ($\sigma = 0.826\,\mathrm{km\,s^{-1}}$), `V_MERID` is north–south antisymmetric ($\sigma = 0.008\,\mathrm{km\,s^{-1}}$), `V_OFFSET` is uniform at $-0.196\,\mathrm{km\,s^{-1}}$ and `V_RESID` has $\sigma = 0.853\,\mathrm{km\,s^{-1}}$. The `V_LIMB` panel is blank because for this line the fit pins e to its lower bound $e = 0$, so the layer is identically zero (the colour bar spans $10^{-9}\,\mathrm{km\,s^{-1}}$); the parameter is flagged `PINNED_AT_ZERO` rather than reported as a measurement.

13 The joint Hα + Fe i product

The standard output of a run that processes both spectral lines is *one figure per observation time*: one row per line and one column per physical component, so that the four panels of a row read directly as the decomposition of the observed velocity field (Figure 4). The implementation is `plotting_grid.py`; it is independent of the CHASE data model and takes only 2-D arrays, extents, limits and labels.

Layout rules. These are fixed in code, so every figure of a series is directly comparable.

1. Rows and columns are separated by the same *absolute* gap in inches (`gap_in`, default 0.048 in); relative spacing such as `wspace`/`hspace` is not used.
2. Every panel is sized from the data aspect, panel width = aspect × panel height with aspect = $(x_1 - x_0)/(y_1 - y_0)$, so `set_aspect('equal')` changes nothing and no extra whitespace appears between columns.
3. Frames are thick (3.2 pt); the panel letter `(a)`, `(b)`, … sits in the upper-left corner and the component name in the upper-right.

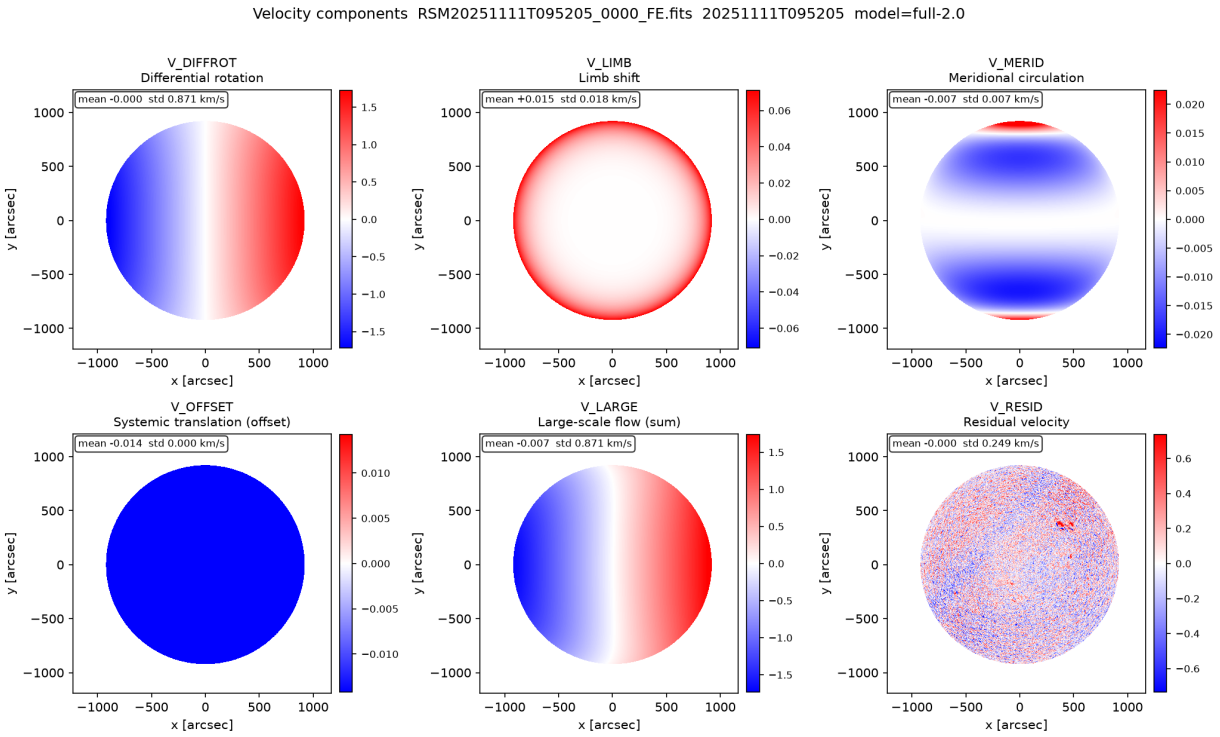


Figure 3: Fe I, 2025-11-11 09:52:05 UT (`RSM20251111T095205_0000_FE.fits`), from the same pipeline with no change other than the input line: the dispersion gives `CALSPEC` = 2.2108 km s^{-1}/pixel against 2.2139 for Hα, and the output is tagged `_FEI`. Here $e = 0.186$ km s^{-1} is a genuine positive fit, so `V_LIMB` shows a real limb-brightening ring rather than a blank panel, and the residual is three times smaller ($\sigma = 0.249$ km s^{-1}). This pair of figures illustrates why identifiability is reported explicitly: the same code flags `e` for Hα and does not flag it for Fe I, while N is flagged `SCAN_GRID` in both cases.

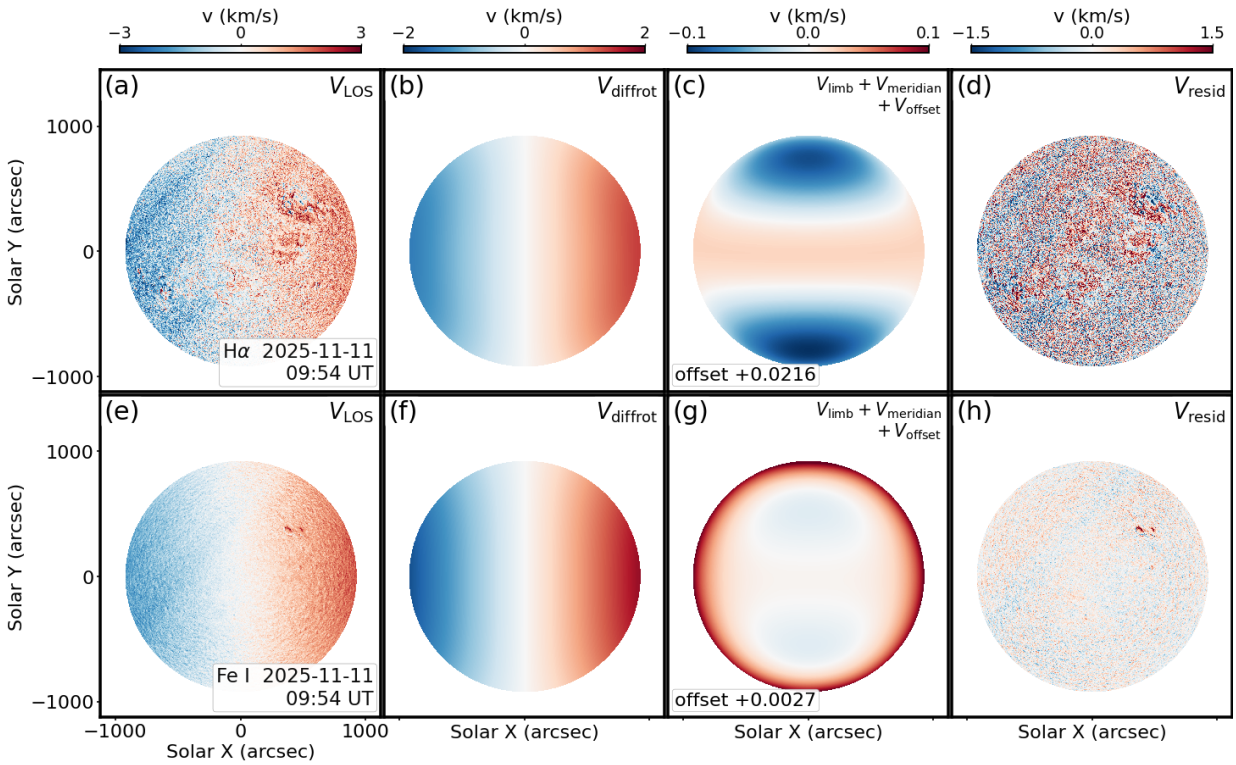


Figure 4: The joint product for 2025-11-11 09:54 UT (20251111T095427). Row 1 is Hα, row 2 is Fe I, and the columns are $V_{\rm LOS}$, $V_{\rm diffrot}$, $V_{\rm limb} + V_{\rm merid} + V_{\rm offset}$ and $V_{\rm resid}$. The median $V_{\rm offset}$ of each line is annotated in the third column (+0.0216 and +0.0027 km s^{-1}), and each row carries its own observing time. The third column is a sum of three small terms and is therefore scaled to the 99.5th percentile of both lines together, while the other three columns keep the fixed ranges ±3, ±2 and ±1.5 km s^{-1} so that the two rows can be compared by eye.

4. One horizontal colour bar per column, placed above the first row, with its title and tick labels above the bar.
5. Axis titles are set on the bottom row and on the first column; tick *labels* appear only on the first column, while tick marks stay on every bottom/left panel, so large fonts never collide across columns.
6. The observing time is written in the lower-right corner of the first panel of each row, and the median V_{offset} is annotated in the third column.
7. The view limits are derived from `R_SUN` and `CDELT1` rather than hard-coded: the half width is $1.155\,R_\odot$ and the top of the frame is $(1+0.495)\,R_\odot$, which at $R_\odot = 970$ arcsec gives $x = \pm 1120$ and $y = (-1120,\, 1450)$ arcsec. The extra room at the top keeps the component name off the disk.
8. The canvas is sized from the font scale, the panel height and the margins, so text is never clipped; at `font_scale` = 2.5, `panel_height_in` = 4.75 the figure is 18.4×13.0 in.

Pairing and invocation. A product is named `<stamp>_<LINE>_scv.fits`, so the observation stamp is the pairing key. Only stamps that carry *both* lines receive a figure, and one figure is written for every such time step. Three entry points are available: `scv -joint-grid` during a directory run, the stand-alone script `scripts/plot_joint_ha_fe.py` over an existing output tree, and the Python functions `plot_joint_grid`, `find_joint_pairs` and `make_joint_series`.

A Symbols

symbol	meaning
B, L, ρ	heliographic latitude, longitude, heliocentric angle
B_0	disk-centre latitude (header `B0`)
$\mu = \cos\rho$	limb-darkening geometrical factor
$R_\odot$	solar radius: pixels via header `R_SUN`; physically 6.957×10^5 km
δ_p	sub-pixel cross-correlation displacement of pixel p (Eq. (13))
κ	velocity calibration, km/s per pixel (Eq. (14))
$\mathcal{P}$, $\mathcal{M}$	LOS projection factors of rotation and meridional circulation (Eqs. (16), (17))
$\boldsymbol{X}, \boldsymbol{\beta}$	design matrix and parameter vector
w	correlation window half-width (`correlation_half_width`)
s	strip height (`spatial_strip_rows`)

B Key configuration options (v2.1 names)

option	default	meaning
`limb_exponent`	`None`	fix the power-law exponent N; default is to scan
`limb_exponent_scan`	$1 \ldots 5$	grid of N values
`limb_amplitude_positive`	`True`	constrain $e \geq 0$
`correlation_half_width`	10	correlation window half-width w
`reference_profile_radius`	100	radius of the reference-profile average (pixels)
`continuum_subtraction`	on outside legacy	subtract the window mean

option	default	meaning
`limit_subpixel_shift`	on outside legacy	clip the sub-pixel term to ± 0.5
`min_spectral_samples`	90	minimum samples for the legacy line-centre method
`disk_mask_radius`	0.95	mask radius in units of $R_\odot$
`active_region_sigma_threshold`	3.0	active-region threshold in σ
`active_region_robust_sigma`	on outside legacy	use a robust (MAD) σ
`min_active_region_area`	100	minimum area of a detected region (pixels)
`active_region_grow_steps`	3	dilation steps for active regions
`active_region_shrink_steps`	1	erosion steps for active regions
`spatial_strip_rows`	32	streaming strip height s
`parallel_processing`	`False`	reserved
`parallel_processes`	1	reserved
`rotation_amplitude`	`'kms'`	with `'rad_per_s'`, A, B, C, D are angular rates
`zero_point_km_s`	0.0	constant zero-point correction
`require_velocity_unit`	`True`	refuse 2-D input whose `BUNIT` is not a velocity